

Programming in Python

Detailed Notes for Grade 10

Introduction

Python is a programming language. To write and execute a Python program, we need a **Python interpreter** installed on the computer, or we can use an online Python interpreter. The interpreter is also called the **Python Shell**.

Python allows us to:

- take input from users,
- process data,
- make decisions,
- repeat tasks,
- store multiple values,
- create reusable functions,
- work with files,
- use libraries,
- draw graphics, and
- represent data using charts and graphs.

1. Input and Output Statements

1.1 Input/Output Statements

I/O statements help us interact with a program.

There are two basic operations:

- **Input** – taking data from the user.
- **Output** – displaying information to the user.

The book introduces `print()` for output and `input()` for input.

1.2 `print()` Statement

The `print()` statement is used to **display output** or data that we want to show to the user.

Syntax

```
print(value)
```

Example

```
print("Hello, Python!")
```

Output

Hello, Python!

The book also demonstrates using `print()` with variables and calculations:

```
x = 3
y = 2
print("Sum of", x, "and", y, "is", x + y)
```

The `print()` statement can therefore display:

- text,
- variables,
- numbers,
- calculations,
- and combinations of these.

Another example

```
name = "Aarav"
age = 15
print("Name:", name)
print("Age:", age)
```

1.3 input() Function

The `input()` function is used to **take input from the user**.

Syntax

```
variable = input("message")
```

Example

```
number = input("Enter number: ")
```

The message "Enter number: " is displayed to the user, and the value entered by the user is stored in `number`.

Important clarification

The value obtained using `input()` is normally treated as a string. Therefore, when we want a number for calculation, we commonly convert it using `int()` or `float()`.

Example:

```
age = int(input("Enter your age: "))
```

For decimal values:

```
price = float(input("Enter price: "))
```

Example from the book

```
x = int(input("Enter first number: "))
y = int(input("Enter second number: "))
print("Sum of", x, "and", y, "is", x + y)
```

2. Data Types, Identifiers and Variables

2.1 Data Types

A **data type** tells us what kind of value a variable contains.

The book discusses four basic data types:

1. Integer
2. Float
3. String
4. Boolean

2.2 Integer (`int`)

An **integer** is a whole number.

It can be:

- positive,
- negative, or
- zero.

Example

-3, -2, -1, 0, 1, 2, 3

Example:

```
age = 15
```

Here, `age` contains an integer value.

The book describes integers as whole numbers ranging from negative infinity to positive infinity.

2.3 Float (`float`)

A **float** is a number containing a decimal value.

Examples

```
3.14  
-0.5  
1.567
```

Example:

```
height = 5.8
```

2.4 String (`str`)

A **string** consists of characters such as:

- alphabets,
- numbers,
- special characters, and
- combinations of these.

The book shows strings enclosed in quotation marks.

Examples

```
"hello"  
"Python@"  
"Bhaktapur1"  
"@##kathmandu"
```

Example:

```
name = "Ram"  
school = "ABC School"
```

Here, `name` and `school` contain string values.

Important point

A number inside quotation marks is treated as a string.

```
age = "15"
```

Here, 15 is text, not an integer.

2.5 Boolean (bool)

A **Boolean** data type contains only two values:

```
True  
False
```

Examples

```
is_student = True  
has_mobile = False
```

Boolean values are commonly used when making decisions.

For example:

```
age = 18  
print(age >= 18)
```

The result is:

```
True
```

2.6 Identifier

An **identifier** is a name given to a program unit such as:

- variables,
- functions,
- classes,
- or other entities.

Examples:

```
age  
student_name  
calculate_area
```

Here, `age`, `student_name`, and `calculate_area` can be identifiers.

Remember

A function name is also an identifier.

Example:

```
def greet():  
    print("Hello")
```

Here, `greet` is an identifier.

2.7 Variable

A **variable** is like a container that holds information.

A variable is created when a value is assigned to it.

Example

```
a = 15
```

Here:

- `a` → variable
- `15` → value

Another example:

```
name = "Sita"  
age = 15
```

The variable `name` stores `"Sita"` and `age` stores `15`.

Easy way to remember

Think of a variable as a **labelled container**.

```
age → 15  
name → "Sita"
```

3. Operators

Operators are special symbols used to perform different actions on numbers and words.

The book covers:

1. Arithmetic operators
2. Relational operators
3. Logical operators

3.1 Arithmetic Operators

Arithmetic operators are used to perform mathematical operations.

Operator	Name	Example	Result
+	Addition	10 + 20	30
-	Subtraction	10 - 20	-10
*	Multiplication	10 * 20	200
/	Division	20 / 10	2
%	Modulus	20 % 10	0
**	Exponent	10 ** 2	100
//	Floor Division	9 // 2	4

1. Addition +

```
a = 10
b = 20
print(a + b)
```

Output:

30

2. Subtraction -

```
print(10 - 20)
```

Output:

-10

3. Multiplication *

```
print(10 * 20)
```

Output:

```
200
```

4. Division /

```
print(20 / 10)
```

Division gives the quotient.

5. Modulus %

The modulus operator gives the **remainder**.

```
print(9 % 2)
```

Output:

```
1
```

This is very useful for checking whether a number is even or odd.

```
if num % 2 == 0:  
    print("Even")  
else:  
    print("Odd")
```

The book uses this type of logic in its function example.

6. Exponent **

It is used for powers.

```
print(2 ** 3)
```

Output:

```
8
```

7. Floor Division //

Floor division returns the integer/floor result of division.

```
print(9 // 2)
```

Output:

4

The book specifically asks students to distinguish / and //.

3.2 Relational Operators

Relational operators are used to **compare values**.

They produce a Boolean result: True or False.

Operator	Meaning	Example
==	Equal to	5 == 5
!=	Not equal to	3 != 5
>	Greater than	7 > 5
<	Less than	3 < 9
>=	Greater than or equal to	8 >= 8
<=	Less than or equal to	4 <= 6

Examples

```
print(5 == 5)
```

Output:

True

```
print(3 != 5)
```

Output:

True

```
print(7 > 5)
```

Output:

True

Relational operators are extremely important in conditional statements.

3.3 Logical Operators

Logical operators are used to **combine conditions** and make decisions based on different situations.

The three logical operators are:

Operator	Meaning	Example
and	AND	a and b
or	OR	a or b
not	NOT	not (a)

and

Both conditions must be true.

Example:

```
age >= 16 and age <= 18
```

or

At least one condition should be true.

Example:

```
day == "Saturday" or day == "Sunday"
```

not

It reverses a Boolean result.

Example:

```
not (True)
```

Result:

```
False
```

4. Conditional Statements

A conditional statement allows a program to make decisions based on a condition.

4.1 if Statement

The `if` statement executes a block of code **only when a condition is true**.

Syntax

```
if condition:
    statement
```

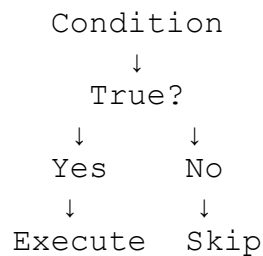
The colon `:` is important.

Example

```
number = int(input("Enter a number: "))
if number > 0:
    print("The number is positive.")
```

If the number is greater than zero, the message is displayed.

Flow



4.2 if-else Statement

The `if-else` statement executes one block when the condition is true and another block when the condition is false.

Syntax

```
if condition:
    statement_when_true
else:
    statement_when_false
```

Example

```
user_age = int(input("How old are you? "))
if user_age >= 18:
    print("You are an adult!")
else:
    print("You are a teenager or a kid.")
```

There are always two possible paths:

- condition true → if block,
- condition false → else block.

4.3 if-elif-else Statement

This structure is used when we need to check **multiple conditions in sequence**.

Syntax

```
if condition1:
    statement1
elif condition2:
    statement2
else:
    statement3
```

Example

```
user_number = int(input("Enter a number: "))
if user_number > 0:
    print("The number is positive.")
elif user_number == 0:
    print("The number is zero.")
else:
    print("The number is negative.")
```

Important point

The program checks the conditions from top to bottom.

The first condition that becomes true is executed.

4.4 Nested if Statement

A **nested if** means an if statement placed inside another if statement.

Example from the book

```
age = int(input("Enter your age: "))

if age >= 16:
    print("You are eligible for citizenship.")
    if age >= 18:
        print("You are eligible to cast a vote.")
    else:
        print("You are not eligible to cast vote.")
else:
    print("You are a minor.")
```

Understanding the example

First, the program checks:

`age >= 16`

Only if this is true does it check:

`age >= 18`

So the second condition depends on the first condition.

Key difference

if-elif-else:

Check condition 1
↓
If false → Check condition 2
↓
If false → else

Nested if:

Check first condition
↓
True
↓
Check another condition

5. Iteration / Looping

Iteration means doing something repeatedly.

The book describes iteration as the process of repeating a particular task until a specified condition is satisfied.

The two fundamental loops are:

1. `for` loop
2. `while` loop

5.1 `for` Loop

A `for` loop is used when we know **how many times** we want to repeat a block of code.

Syntax

```
for item in sequence:
    statement
```

Example

```
for x in range(5):
    print("Programming")
```

This prints "Programming" five times.

`range()`

The `range()` function is commonly used with `for` loops.

Example:

```
for i in range(1, 11):
    print(i)
```

This prints:

```
1
2
3
...
10
```

The book also gives:

```
for i in range(2, 21, 2):  
    print(i)
```

This prints even numbers from 2 to 20.

5.2 while Loop

A while loop repeatedly executes a block of code **as long as a condition remains true**.

Syntax

```
while condition:  
    statement
```

Example

```
count = 1  
  
while count <= 5:  
    print(count)  
    count += 1
```

Output:

```
1  
2  
3  
4  
5
```

Important point

The value controlling the condition should normally be changed inside the loop.

For example:

```
count += 1
```

Otherwise, the condition may remain true and the loop may continue indefinitely.

for Loop vs while Loop

for loop	while loop
Usually used when the number of repetitions is known	Used when repetition depends on a condition
Often used with <code>range()</code>	Works directly with a condition
Example: print 1 to 10	Example: repeat while number is less than 10

The book specifically asks when a `for` loop would typically be used instead of a `while` loop.

6. Python List

A **Python list** is a built-in data type used to store **multiple data items in a single variable**.

Example

```
thislist = ["Computer", "Science", 20, True]
print(thislist)
```

A list can contain different types of values, as shown in the book:

- "Computer" → string
- "Science" → string
- 20 → integer
- True → Boolean

Creating a List

A list is written using square brackets:

```
students = ["Ram", "Sita", "Hari"]
```

Another example:

```
marks = [75, 80, 90, 65]
```

Basic List Operations

The book only gives the basic idea of creating, storing, and printing a list. For better practical understanding, these basic operations can be remembered.

Accessing an item

List positions start from 0.

```
students = ["Ram", "Sita", "Hari"]  
print(students[0])
```

Output:

Ram

Adding an item

```
students.append("Gita")
```

Changing an item

```
students[0] = "Aarav"
```

Finding the number of items

```
len(students)
```

Looping through a list

```
for student in students:  
    print(student)
```

Important idea

A list is useful when several related values need to be stored under one variable name.

Example:

```
marks = [70, 85, 90, 78, 88]
```

7. Functions

A **function** is a block of organized, reusable code used to perform a single, related action.

Functions provide:

- better modularity, and
- code reusability.

Instead of writing the same code again and again, we can put it inside a function and call it whenever necessary.

7.1 Types of Functions

The book divides Python functions into two types:

1. Built-in functions
2. User-defined functions

Built-in Functions

Built-in functions are functions already available in Python.

Examples given by the book include:

```
print()  
int()  
len()  
sum()
```

These functions are available when the Python interpreter starts.

Example:

```
print("Hello")  
numbers = [1, 2, 3]  
print(len(numbers))
```

7.2 User-defined Functions

Functions created by the programmer to perform a specific task are called **user-defined functions**.

7.3 Creating a User-defined Function

A function definition begins with the keyword:

```
def
```

def is short for **define**.

General syntax

```
def function_name(parameters) :  
    statements  
    return value
```

The parameters and return statement are optional.

Therefore, a function:

- may have parameters or may not,
- may return a value or may not.

Important points

1. Function header ends with :.
2. Function name should be unique.
3. Rules for naming identifiers also apply to function names.
4. Indentation is important.
5. Statements outside the indentation are not part of the function.

Example

```
def add_numbers(x, y):  
    sum = x + y  
    return sum
```

Calling the function:

```
num1 = 5  
num2 = 6  
  
print("The sum is", add_numbers(num1, num2))
```

7.4 Scope of Functions

Scope refers to the part of a program where a function or variable can be accessed.

The book discusses:

1. Local scope
2. Global scope

Local Scope

Variables declared inside a function are accessible only within that function.

```
def show():  
    x = 10  
    print(x)
```

Here, x is a local variable.

The book notes that a local variable exists while the function is executing.

Global Scope

A variable declared outside a function can be accessed from different parts of the program.

Example:

```
x = 10

def show():
    print(x)

show()
```

Here, x is a global variable.

Easy comparison

Local variable	Global variable
Declared inside a function	Declared outside a function
Used inside that function	Can be accessed from different parts of the program
Limited scope	Wider scope

7.5 Function Return Value

The `return` statement specifies the value that should be sent back to the calling part of the program.

Syntax

```
def function_name(arguments):
    return value
```

Example

```
def area(length, width):
    return length * width

length = 5
width = 8

result = area(length, width)

print("Area of Rectangle:")
print(result)
```

Here:

1. `area()` receives `length` and `width`.
2. It calculates the area.
3. `return` sends the result back.
4. The returned result is stored in `result`.

7.6 Parameters and Arguments

These two terms are very important.

Parameter

A **parameter** is a value/name written in the parentheses when defining a function.

Example:

```
def area(length, width):
```

Here:

- `length` → parameter
- `width` → parameter

Argument

An **argument** is the actual value passed to a function when it is called.

Example:

```
area(5, 8)
```

Here:

- `5` → argument
- `8` → argument

The book states that an argument can be a:

- constant,
- variable, or
- expression.

Easy way to remember

```
def add(a, b):
```

a and b are **parameters**.

```
add(5, 10)
```

5 and 10 are **arguments**.

7.7 Passing Parameters or Arguments

Python supports three types of formal arguments:

1. Positional / Required arguments
2. Default arguments
3. Keyword / Named arguments

1. Positional / Required Arguments

The number of arguments should match the number of required parameters.

Example:

```
def check(a, b, c):  
    ...
```

Valid calls include:

```
check(x, y, z)  
check(2, x, y)  
check(2, 5, 7)
```

The values are matched according to their position.

2. Default Arguments

Sometimes a parameter has a value that is already known.

A default argument allows us to assign a default value to a parameter.

The book gives interest calculation as an example, where the rate may normally be 10%.

Example:

```
def interest(prin, time, rate=0.10):  
    ...
```

Here:

```
rate = 0.10
```

is the default value.

The book also gives examples showing that required parameters should come before default parameters:

```
def Interest(prin, time, rate=0.10):
```

is valid.

```
def Interest(prin, time=2, rate):
```

is not valid because a required parameter is placed after a default parameter.

Book sample

```
def simple_interest(principal, time, rate=10):  
    return (principal * time * rate) / 100
```

```
p = float(input("Enter Principal Amount: "))  
t = float(input("Enter Time in Years: "))
```

```
print("Simple Interest:", simple_interest(p, t))
```

3. Keyword / Named Arguments

Keyword arguments allow us to specify the parameter name while calling the function.

This means the order does not have to match the original parameter order.

For example, a function could be called using named parameters such as:

```
interest(prin=2000, time=2, rate=0.10)
```

or:

```
interest(time=2, rate=0.12, prin=2000)
```

The book explains that these calls remain valid even though the order differs because the parameter names are specified.

7.8 Returning Values from Functions

Functions may or may not return a value.

The book classifies them as:

1. Non-void functions
2. Void functions

Non-void Function

A function that returns a computed result is called a **non-void function** in the book.

The value is returned using:

```
return <value>
```

A returned value can be:

1. A literal
2. A variable
3. An expression

Example

```
def add(x, y):  
    return x + y
```

Here:

```
x + y
```

is an expression.

Literal

A function can return a literal:

```
def message():  
    return "Hello"
```

Variable

```
def result():  
    x = 10  
    return x
```

Expression

```
def square(n):  
    return n * n
```

Void Function

A **void function** performs some action but does not return a computed value to the caller.

The book explains that a void function may have a `return` statement without a value:

```
return
```

Example:

```
def greet():  
    print("Hello")
```

Another example:

```
def quote():  
    print("Python is good")  
    return
```

The main action is performed inside the function, but no computed value is sent back.

7.9 Returning Multiple Values

Python allows a function to return multiple values separated by commas.

Example

```
def squared(x, y, z):  
    return x * x, y * y, z * z
```

The returned values can be stored together:

```
t = squared(2, 5, 7)  
print(t)
```

They can also be assigned to separate variables:

```
def squared(x, y, z):  
    return x * x, y * y, z * z
```

```
v1, v2, v3 = squared(2, 3, 4)

print(v1, v2, v3)
```

Another book example

```
def square_and_cube(n):
    return n ** 2, n ** 3

num = int(input("Enter a number: "))

sq, cb = square_and_cube(num)

print("Square:", sq, "Cube:", cb)
```

8. Python Libraries and Packages

8.1 Module

The book explains that smaller, manageable units created in programming are called **modules**.

A library is related to modules.

8.2 Python Library

A **library** is a collection of modules and packages that provide functionality for particular applications or requirements.

The book gives examples such as:

NumPy

A library for scientific computing that supports large arrays and mathematical functions.

Pandas

Helps with data and provides tools for working with different kinds of data.

Matplotlib

Used for making charts and is often used with Pandas.

8.3 Python Package

A **package** is described in the book as a container containing various functions for specific tasks.

For example, the Math package includes functions such as:

```
sqrt()
```

for finding the square root of a number.

Packages help with **code reusability**.

To access modules or files from a package, we use the `import` statement.

The dot operator `.` is used to access functions from modules.

9. Importing and Using Standard Libraries

Python has standard libraries containing code and functionality for common tasks such as:

- mathematics,
- file handling,
- system operations,
- and others.

We use the `import` statement to use them.

9.1 Import the Entire Module

Syntax

```
import module_name
```

Example

```
import math  
  
print(math.sqrt(25))
```

Output:

```
5.0
```

The module name is written before the function name:

```
math.sqrt()
```

9.2 Import a Specific Function

We can import only the function that we need.

```
from math import sqrt  
  
print(sqrt(25))
```

There is no need to write `math.sqrt()` because `sqrt` has been imported directly.

9.3 Import a Module with an Alias

An alias provides a shorter name for a module.

Example

```
import datetime as dt
```

Now `dt` can be used as a shortcut for `datetime`.

The book gives:

```
print(dt.datetime.now())
```

Another common example from the book is:

```
import pandas as pd
```

Here:

```
pd
```

is the alias for `pandas`.

9.4 Import All Functions from a Module

The book also shows:

```
from math import *
```

This imports all functions from the module.

For example:

```
from math import *  
  
print(sin(90))
```

For Grade 10, students should remember the syntax and concept rather than using this form everywhere.

10. Introduction to Popular Python Libraries

10.1 math Library

The `math` library is a built-in Python library that provides mathematical functions.

It includes functions for:

- mathematical constants,
- trigonometry,
- logarithms,
- powers,
- and other mathematical operations.

Examples

```
import math

print(math.sqrt(16))
print(math.sin(math.pi / 2))
```

- `math.pi`
- `sin()`
- `cos()`
- `tan()`
- `log()`
- `log10()`
- `pow()`

10.2 random Library

The `random` module is used to:

- generate random numbers,
- make random selections,
- perform random sampling,
- generate values for games and simulations.

Important functions include:

`randrange()`

Returns a random number within a range.

randint()

Returns a random integer within a range.

Example:

```
import random  
  
print(random.randint(1, 10))
```

choice()

Returns a random item from a list, tuple, or string.

```
print(random.choice(["apple", "banana", "Orange"]))
```

random()

Generates random floating-point numbers.

10.3 pandas Library

Pandas provides tools for:

- manipulating data,
- analyzing data,
- managing large datasets,
- data cleaning,
- reading data,
- writing data,
- and working with tables.

It supports formats such as:

- CSV,
- Excel,
- SQL,
- and others.

Installing Pandas

```
pip install pandas
```

Common Pandas features in the book

read_csv()

Loads CSV data into a table-like structure called a **DataFrame**.

```
pd.read_csv("data.csv")
```

DataFrame.info()

Displays information such as:

- column names,
- data types,
- missing values.

DataFrame.shape

Returns the size of the DataFrame as:

```
(rows, columns)
```

pandas.DataFrame()

Creates a DataFrame with rows and columns.

Example:

```
import pandas as pd

data = {
    "Name": ["Shyam", "Sanskar"],
    "Age": [25, 30]
}

df = pd.DataFrame(data)
print(df)
```

10.4 turtle Library

Turtle is a pre-built Python module used to create graphical representations using a cursor called a **turtle**.

It allows programmers to create:

- shapes,
- figures,

- lines,
- designs,
- and simple animations.

The book also describes turtle as useful for learning programming through interactive graphics.

Example:

```
import turtle

s = turtle.Turtle()

for i in range(4):
    s.forward(50)
    s.right(90)

turtle.done()
```

This program can be used to draw a square.

10.5 matplotlib Library

Matplotlib is a Python graph plotting library used for high-quality visualizations and graphs.

The book mentions that it can create:

- line plots,
- scatter plots,
- histograms,
- bar charts,
- pie charts,
- box plots,
- and 3-dimensional plots.

Installation

```
pip install matplotlib
```

Basic import

```
import matplotlib.pyplot as plt
```

11. Graphics Using Turtle

Turtle allows us to draw using Python instructions.

It is especially useful for understanding:

- loops,
- functions,
- variables,
- movement,
- and graphical designs.

The book describes turtle as a simple and fun way to learn programming, with benefits including visualization, interactive learning, creativity, and easier debugging.

11.1 **forward(distance)**

Moves the turtle forward by a specified distance.

It can also be written as:

```
turtle.fd(distance)
```

Example:

```
import turtle

sk = turtle.Turtle()
sk.forward(50)

turtle.done()
```

11.2 **back(distance)**

Moves the turtle backward.

Other forms:

```
turtle.bk(distance)
turtle.backward(distance)
```

The movement does not change the turtle's heading.

Example:

```
import turtle

sk = turtle.Turtle()
sk.backward(50)

turtle.mainloop()
```

11.3 right (angle)

Turns the turtle clockwise by the specified angle.

Short form:

```
turtle.rt(angle)
```

Example:

```
import turtle

t = turtle.Turtle()

t.right(30)

turtle.mainloop()
```

11.4 left (angle)

Turns the turtle counterclockwise by a specified angle.

Short form:

```
turtle.lt(angle)
```

Example:

```
import turtle

t = turtle.Turtle()

t.left(100)

turtle.mainloop()
```

11.5 penup ()

`penup()` lifts the turtle's pen from the drawing surface.

When the turtle moves after `penup()`, it does not draw.

Example:

```
turtle.penup()  
turtle.forward(30)
```

The turtle moves without drawing.

To start drawing again, `pendown()` is used.

11.6 color()

The `color()` method changes the color of the turtle's pen.

The default color is black.

Example:

```
import turtle  
  
turtle.forward(50)  
  
turtle.color("blue")  
turtle.forward(150)  
  
turtle.color("red")  
turtle.forward(50)
```

11.7 fillcolor()

`fillcolor()` selects the color used to fill a shape.

The book explains that the color can be a color name or a hexadecimal color value.

Example:

```
t.fillcolor("blue")
```

11.8 begin_fill()

This tells turtle to start filling a closed shape.

```
t.begin_fill()
```

11.9 end_fill()

This tells turtle to stop filling the shape.

```
t.end_fill()
```

Example:

```
import turtle

t = turtle.Turtle()

t.fillcolor("blue")

t.begin_fill()
t.circle(50)
t.end_fill()

turtle.done()
```

The book demonstrates this concept using user input for radius and color.

11.10 shape()

The `shape()` method changes the turtle's shape.

Syntax

```
turtle.shape(name=None)
```

Available shapes mentioned in the book include:

- "arrow"
- "turtle"
- "circle"
- "square"
- "triangle"
- "classic"

Example:

```
import turtle
turtle.shape("circle")
```

```
turtle.forward(100)
```

The book also demonstrates changing the shape between circle, triangle, square, arrow, and turtle.

12. Errors and Exceptions

Errors are an inevitable part of programming.

Error handling helps programmers identify, manage, and resolve problems so that programs become more reliable and maintainable.

The chapter covers:

1. Syntax errors
2. Runtime errors
3. Logical errors
4. Exceptions
5. Exception handling

12.1 Syntax Error

A **syntax error** occurs when the code violates the rules of the programming language.

For example:

- missing colon,
- incorrect syntax,
- incorrect structure.

Example of an incorrect statement:

```
if age >= 18
    print("Adult")
```

The colon is missing after the condition.

Correct form:

```
if age >= 18:
    print("Adult")
```

12.2 Runtime Error

A runtime error occurs while the program is being executed.

The book gives **division by zero** as an example.

Example:

```
numerator = int(input("Enter numerator: "))
denominator = int(input("Enter denominator: "))

result = numerator / denominator

print(result)
```

If the user enters 0 as the denominator, an error occurs during execution.

12.3 Logical Error

A logical error occurs when:

- the program runs,
- there is no syntax error,
- there may be no runtime error,
- but the output is incorrect.

The problem is with the logic or algorithm used.

Simple example

Suppose we want the average of two numbers but accidentally write:

```
result = (a + b) / 3
```

The program runs, but the answer is wrong.

The error is logical.

12.4 Exception

An exception occurs when a syntactically correct statement causes an error during execution.

The book gives:

```
divide_by_zero = 7 / 0
```

as an example.

Division by zero is not possible, so Python produces an exception.

12.5 Exception Handling

Exception handling allows us to anticipate possible problems and provide alternative solutions or meaningful warning messages instead of allowing the program to stop unexpectedly.

The book introduces:

- `try`
- `except`
- `else`
- `finally`

12.6 `try` Block

The `try` block contains code that we want Python to test for errors.

If an exception occurs, execution moves to the `except` block.

Example structure

```
try:
    risky_code
```

12.7 `except` Block

The `except` block handles the error or exception.

If the code inside `try` produces an exception, Python executes the `except` block.

Example

```
try:
    a = 10 / 0
except:
    print("An error occurred.")
```

12.8 `else` Block

The `else` block is optional.

It runs **only when no exception occurs** in the `try` block.

Example:

```
try:
    a = 10 / 2
except:
    print("Error")
else:
    print("Calculation successful")
```

12.9 finally Block

The **finally** block **always executes**, whether an exception occurs or not.

It is commonly used for cleanup operations such as closing files or releasing resources.

Example:

```
try:
    file = open("test.txt", "r")
except:
    print("File could not be opened.")
finally:
    print("Program finished.")
```

Easy Summary of Exception Handling

```
try
↓
Try to execute the code
↓
Error?
├─ Yes → except
└─ No  → else
↓
finally
```

Remember:

Block	Purpose
try	Tests code for errors
except	Handles the error
else	Runs when there is no error
finally	Always runs

13. File Handling Using Python/Pandas

A **file** is a collection of data stored on a disk and identified by a filename.

Files allow data to be stored:

- temporarily, and
- permanently.

Python provides functions and methods for:

- creating files,
- opening files,
- reading files,
- writing files,
- closing files.

The chapter also explains that Pandas makes importing and analyzing data easier and supports file formats such as:

- CSV,
- Excel,
- JSON,
- and others.

13.1 Benefits of File Handling

The book lists four major benefits.

1. Versatility

File handling supports many operations such as:

- creating,
- reading,
- writing,
- appending,
- renaming,
- deleting files.

2. Flexibility

Python can work with different file types, including:

- text files,
- binary files,
- CSV files.

It also supports operations such as read, write and append.

3. User-friendly

Python provides an easy interface for creating, reading, and manipulating files.

4. Cross-platform

Python file-handling functions work across operating systems such as:

- Windows,
- Mac,
- Linux.

13.2 Difficulties of File Handling

The book identifies four difficulties.

1. Error-prone

Problems can occur due to:

- incorrect code,
- file permissions,
- file locks,
- and other file-system issues.

2. Security Risks

A program can become risky if user input is used to access or modify sensitive files.

3. Complexity

Advanced file formats and operations can become complex.

4. Performance Issues

File operations can become slower when working with very large files or complex operations.

13.3 Modes of File Handling

Python provides different file modes.

The three main modes emphasized in the syllabus are:

- `r` → Read
- `w` → Write

- a → Append

Read Mode – r

Read mode is used to read a file.

```
file = open("Test.txt", "r")
```

The file cannot be changed through this mode.

If the file does not exist, an error occurs.

Write Mode – w

Write mode is used to write data.

```
file = open("Test.txt", "w")
```

If the file does not exist, it is created.

If the file already exists, its previous contents are overwritten.

Append Mode – a

Append mode is used to add new data to the existing file.

```
file = open("Test.txt", "a")
```

If the file does not exist, it is created.

If it already contains data, the new data is added instead of replacing the old data.

13.4 Reading a File

The `read()` method reads the entire content of a file.

Example:

```
file = open("Test.txt", "r")
```

```
content = file.read()
```

```
print(content)
```

```
file.close()
```

File Reading Methods

read()

Reads the complete content.

```
file.read()
```

readline()

Reads one line.

```
file.readline()
```

readlines()

Reads lines in the form of a collection of lines.

```
file.readlines()
```

read(size)

Reads a specified number of characters.

Example:

```
file.read(10)
```

The book demonstrates `readline()`, `read(10)`, and `read()`.

13.5 Creating a New File

The book gives two modes for creating a new file:

x

Creates a new file.

If a file with the same name already exists, an error occurs.

```
file = open("nl.txt", "x")
```

w

Creates a new file if necessary.

If the file already exists, it can overwrite the existing content.

```
file = open("nl.txt", "w")
```

13.6 Writing to a File

The `write()` method writes a string to a file.

Example:

```
file = open("Test.txt", "w")

file.write("Hello, World!")

file.close()
```

Important

When using `w` mode:

- old contents can be erased,
- new content is written.

13.7 `write()` Method

`write()` is used to write a string.

Example:

```
with open("file.txt", "w") as nl:
    nl.write("This is the first line\n")
    nl.write("This is the second line\n")
    nl.write("This is the third line")
```

13.8 `writelines()` Method

`writelines()` allows multiple strings to be written.

Example:

```
nl = open("file.txt", "w+")

nl.writelines([
    "This is the first line\n",
    "This is the second line\n",
```

```
        "This is the third line\n"
    ])\n\nnl.close()
```

13.9 Closing a File

After completing file operations, the file should be closed.

The `close()` method terminates the file-related resources and frees the system resources.

The book emphasizes that closing a file is good practice.

Example:

```
file = open("nl.txt", "r")\nprint(file.read())\nfile.close()
```

14. Reading and Writing CSV Files

14.1 What is CSV?

CSV stands for **Comma Separated Values**.

CSV is one of the most common formats for importing and exporting data from:

- spreadsheets,
- databases,
- and other programs.

Normally, commas separate individual values.

Example:

```
Name, Age, Class\nRam, 15, 10\nSita, 16, 10
```

Advantages of CSV

The book gives three major advantages.

Portability

CSV files are plain text and can easily be opened in:

- text editors,
- Excel,
- Google Sheets,
- and similar programs.

Simplicity

CSV has a straightforward structure and is easy to generate and process.

Wide Support

Almost every programming language supports CSV files.

14.2 Using the `csv` Module

Python has a built-in `csv` module.

Import

```
import csv
```

It provides functionality for reading and writing CSV data.

Example

```
import csv

with open("data.csv", "r") as csvfile:
    csv_reader = csv.reader(csvfile)

    for row in csv_reader:
        print(row)
```

14.3 Read CSV Using Pandas

Pandas provides a convenient way to read and manipulate CSV data.

Step 1: Import Pandas

```
import pandas as pd
```

Step 2: Read the CSV file

```
data = pd.read_csv("Salary_Data.csv")
```

The book gives the basic syntax as:

```
pandas.read_csv(filename, delimiter=',')
```

The result is stored in a DataFrame.

14.4 Accessing Column Names

The `.columns` attribute is used to obtain the field/header names.

```
data.columns
```

Accessing a Column

The book demonstrates:

```
data.Salary
```

This accesses the `Salary` column.

Another common form is:

```
data["Salary"]
```

14.5 Writing a CSV File Using Pandas

The DataFrame provides the `to_csv()` method to write data to a CSV file.

Step 1: Import Pandas

```
import pandas as pd
```

Step 2: Create DataFrame

```
header = ["Name", "M1 Score", "M2 Score"]
```

```
data = [  
    ["Sanskar", 62, 80],  
    ["Sambriddhi", 45, 56],  
    ["Aurora", 85, 98]  
]
```

```
data = pd.DataFrame(data, columns=header)
```

Step 3: Write to CSV

```
data.to_csv("Stu_data.csv", index=False)
```

The book explains that:

- comma is the default separator,
- `index=False` removes index numbers from the CSV file.

Example

```
import pandas as pd

data = {
    "Product": ["Laptop", "Smartphone", "Tablet"],
    "Price": [75000, 15000, 20000],
    "Quantity": [3, 10, 5]
}

df = pd.DataFrame(data)

df.to_csv("products.csv", index=False)

print("Data written to products.csv successfully.")
```

15. Data Visualization

Data visualization means presenting data using charts or graphs so that it is easier to understand.

It helps users identify:

- patterns,
- trends,
- connections,
- and ideas.

Importance of Data Visualization

Data visualization helps people understand large amounts of data more easily.

Instead of looking at many rows and columns of numbers, we can use a chart to see information visually.

It helps:

- identify trends,
- recognize patterns,
- communicate ideas,
- understand data,
- and support better decisions.

The book notes that businesses and professionals use visualization to help make better decisions because a huge amount of data is created every day.

Popular Python Visualization Libraries

The book introduces:

1. Matplotlib
2. Seaborn
3. Plotly

15.1 Matplotlib

Matplotlib is one of the most popular Python plotting libraries.

It provides significant freedom and customization, although this may require more code.

Installation

```
pip install matplotlib
```

Import

```
import matplotlib.pyplot as plt
```

Scatter Plot

A **scatter plot** uses dots to represent the relationship between variables.

Matplotlib provides:

```
scatter()
```

Example

```
import pandas as pd
import matplotlib.pyplot as plt

dataset = pd.read_csv("Stu_data.csv")
```

```
plt.scatter(dataset["Name"], dataset["Marks"])

plt.title("Scatter Plot")
plt.xlabel("Name")
plt.ylabel("Marks")

plt.show()
```

Important functions

```
plt.scatter()
plt.title()
plt.xlabel()
plt.ylabel()
plt.show()
```

Bar Chart

A bar chart uses rectangular bars to represent data categories.

The length/height of the bars represents the corresponding data values.

Example

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("tips.csv")

plt.bar(data["total_bill"], data["day"])

plt.title("Bar Chart")
plt.xlabel("Day")
plt.ylabel("Tip")

plt.show()
```

15.2 Seaborn

Seaborn is a Python library used to make beautiful and informative charts.

It is built on top of Matplotlib and works well with Pandas data.

It is useful for charts showing:

- patterns,
- relationships,
- trends.

The book specifically lists:

- line plots,
- bar plots,
- heatmaps.

Installation

```
pip install Seaborn
```

Seaborn Line Plot

The `lineplot()` method is used for a line plot.

Example:

```
import pandas as pd
import seaborn as sn
import matplotlib.pyplot as plt

dataset = pd.read_csv("Stu_data.csv")

sn.lineplot(
    x="Name",
    y="Marks",
    data=dataset
)
plt.show()
```

Heatmap

A heatmap is another visualization supported by Seaborn.

It represents information using different shades or intensities, allowing patterns in data to be seen visually.

The Grade 10 book lists heatmaps as one of Seaborn's important visualization types.

15.3 Plotly

Plotly is an open-source Python library used to make **interactive charts**.

It can create:

- 3D charts,
- scientific and statistical charts,
- financial charts,
- and other interactive visualizations.

Plotly graphs can be viewed in:

- Jupyter notebooks,
- web browsers,
- and HTML files.

Users can interact with charts by:

- clicking,
- zooming,
- and editing the charts.

Installation

```
pip install plotly
```

Plotly Scatter Plot

Plotly Express provides a method for creating scatter plots.

Example from the book:

```
import pandas as pd
import plotly.express
dataset = pd.read_csv("tips.csv")
graph = plotly.express.scatter(
    dataset,
    x="total_bill",
    y="size",
    color="smoker"
)
graph.show()
```

Plotly Line Chart

Plotly Express uses:

```
px.line()
```

Example:

```
import plotly.express as px
import pandas as pd
data = pd.read_csv("stu_data.csv")
fig = px.line(
    data,
    y="Name",
    color="Gender"
)
fig.show()
```

Plotly Bar Chart

Plotly Express uses:

```
px.bar()
```

Example:

```
import plotly.express as px
import pandas as pd
data = pd.read_csv("Stu_data.csv")
fig = px.bar(
    data,
    x="Name",
    y="Marks",
    color="Gender"
)
fig.show()
```

Comparison of Visualization Libraries

Library	Main purpose	Examples
Matplotlib	General plotting and charts	Scatter, Bar
Seaborn	Attractive and informative statistical charts	Line, Bar, Heatmap
Plotly	Interactive and dynamic charts	Scatter, Line, Bar

This comparison follows the roles described in the Grade 10 book.